

Chuỗi và so khớp (Strings and Pattern Matching)

PHỤ THUỘC B/06 cho ý tưởng không tính lại; B/08 cho băm và nghịch lý ngày sinh; B/09 cho cây. Đây là chương cuối của phần B.

TÓM TẮT

Chuỗi là mảng, nhưng có một tính chất riêng làm nó đáng có chương riêng: các đoạn con của nó *chồng lấn* nhau, và sự chồng lấn ấy vừa là nguồn của mọi khó khăn vừa là nguồn của mọi lời giải nhanh. Chương đi từ bài toán so khớp một mẫu — KMP và hàm Z, cả hai đều dựa trên ý “thất bại ở đây cho ta thông tin, đừng vứt đi” — tới nhiều mẫu cùng lúc với Aho–Corasick, tới các cấu trúc nhìn được toàn bộ chuỗi con là trie và mảng hậu tố. Băm chuỗi được đặt cạnh chúng như một lựa chọn ngắn và mạnh với cái giá xác suất. Nếu chỉ nhớ một điều: mọi thuật toán chuỗi nhanh đều trả lời cùng một câu hỏi — *khi so khớp thất bại tại vị trí i , tôi đã biết được gì mà không phải quay lại từ đầu?*

KIẾN THỨC CẦN CÓ TRƯỚC

- **tiền tố, hậu tố** — đoạn đầu và đoạn cuối của một chuỗi. Chuỗi abc có tiền tố a, ab, abc.
- **biên (border)** — một chuỗi vừa là tiền tố vừa là hậu tố thực sự của chuỗi đã cho; ví dụ aba có biên a. Khái niệm trung tâm của KMP.
- **hàm tiền tố π** — $\pi[i]$ là độ dài biên dài nhất của tiền tố kết thúc tại i . Đây là toàn bộ nội dung “tiền xử lý” của KMP.
- **hàm Z** — $z[i]$ là độ dài tiền tố chung dài nhất giữa chuỗi và hậu tố bắt đầu tại i . Cùng sức mạnh với π , thường dễ nghĩ hơn.
- **trie** — cây trong đó mỗi cạnh mang một ký tự, nên mỗi đường từ gốc biểu diễn một tiền tố; dùng để lưu một tập chuỗi và tra theo tiền tố.
- **mảng hậu tố (suffix array)** — mảng các vị trí hậu tố, sắp theo thứ tự từ điển của các hậu tố; cùng với mảng LCP, nó biến nhiều bài chuỗi thành bài mảng thông thường.

CÂU HỎI MỞ ĐẦU

- Q1. Thuật toán so khớp ngây thơ chạy $O(nm)$. KMP chỉ dùng thêm một mảng số nguyên mà xuống $O(n + m)$. Thông tin nào đã bị lãng phí trong cách ngây thơ?
- Q2. Nếu băm chuỗi làm được gần hết những gì KMP làm và code ngắn hơn nhiều, vì sao vẫn phải học KMP?
- Q3. Vì sao mảng hậu tố lại được coi là “giải quyết một họ bài” chứ không chỉ là một cấu trúc?
- Q4. Aho–Corasick tìm đồng thời hàng nghìn mẫu trong một lượt quét. Nó khác gì so với chạy KMP nhiều lần, và cái gì làm cho khác biệt đó có thể xảy ra?
- Q5. Bài toán tìm chuỗi con đối xứng dài nhất có lời giải $O(n)$. Ý tưởng nào cho phép bỏ qua phần lớn công việc?

Chuỗi ký tự là mảng, và mọi kỹ thuật ở chương B/06 đều dùng được. Nhưng có một điều làm chuỗi khác hẳn: một chuỗi độ dài n có $n(n+1)/2$ đoạn con, và các đoạn ấy chồng lên nhau chằng chịt. Điều này vừa là tin xấu — không thể liệt kê hết — vừa là tin tốt, vì sự chồng lấn nghĩa là có

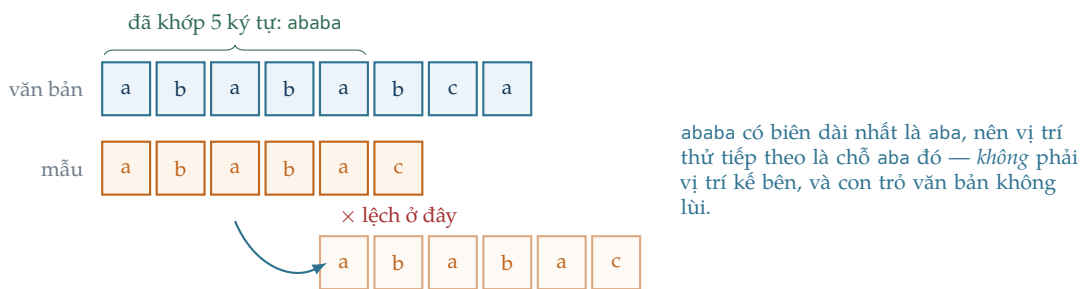
rất nhiều thông tin lặp lại, và mọi thuật toán chuỗi nhanh đều sống nhờ việc khai thác chỗ lặp lại đó.

Bài toán mở đầu là bài toán so khớp: tìm mọi vị trí xuất hiện của mẫu P độ dài m trong văn bản T độ dài n . Cách ngây thơ thử mọi vị trí bắt đầu và so từng ký tự, cho $O(nm)$. Cái lãng phí nằm ở chỗ: khi so khớp thất bại sau khi đã khớp được k ký tự, ta *đã biết* k ký tự đó của văn bản là gì — nhưng cách ngây thơ vứt bỏ thông tin ấy và bắt đầu lại từ vị trí kế tiếp. Toàn bộ chương này là các cách khác nhau để không vứt.

1.1 KMP: dùng lại thông tin của lần thất bại

Giả sử ta đang so mẫu P với văn bản và đã khớp được k ký tự thì gặp ký tự không khớp. Ta biết rằng k ký tự vừa qua của văn bản đúng bằng $P[1..k]$. Câu hỏi đúng là: *vị trí bắt đầu tiếp theo nào còn có cơ hội?* Một vị trí như vậy chỉ khả dĩ nếu phần mẫu tính từ đó khớp với đuôi của $P[1..k]$ — nghĩa là ta cần một chuỗi vừa là tiền tố vừa là hậu tố của $P[1..k]$. Đó chính là khái niệm *biên*.

Vậy toàn bộ việc tiền xử lý của KMP là tính hàm π : với mỗi tiền tố của mẫu, độ dài biên dài nhất của nó. Khi thất bại sau k ký tự khớp, ta không lùi con trỏ văn bản chút nào; ta chỉ nhảy con trỏ mẫu về $\pi[k]$ và so tiếp.



Hình 1.1. Ý tưởng của KMP. Khi lệch sau 5 ký tự khớp, ta biết văn bản vừa qua là ababa. Vị trí bắt đầu tiếp theo có cơ hội phải là chỗ mà mẫu khớp với một hậu tố của ababa đồng thời là tiền tố của mẫu — tức một biên. Biên dài nhất là aba, nên ta trượt mẫu đúng hai bước, và không ký tự nào của văn bản bị đọc lại.

Chi phí $O(n + m)$, và lập luận lại chính là lập luận khấu hao quen thuộc: con trỏ văn bản chỉ tiến, còn con trỏ mẫu tăng tối đa n lần tổng cộng nên cũng chỉ giảm tối đa n lần. Cùng dạng lập luận với hai con trỏ (B/06) và ngăn xếp đơn điệu (B/07) — đây là lần thứ ba nó xuất hiện, và đó không phải trùng hợp.

Hàm Z là công cụ tương đương về sức mạnh nhưng thường dễ nghĩ hơn: $z[i]$ là độ dài đoạn khớp dài nhất giữa chuỗi và hậu tố bắt đầu tại i . Tính được trong $O(n)$ bằng cách duy trì đoạn khớp phải nhất đã biết và tái sử dụng nó. Mẹo dùng chung cho cả hai: để tìm P trong T , hãy tính hàm trên chuỗi ghép $P + \# + T$ với $\#$ là ký tự không xuất hiện ở đâu cả; mọi vị trí có giá trị bằng m chính là một lần xuất hiện.

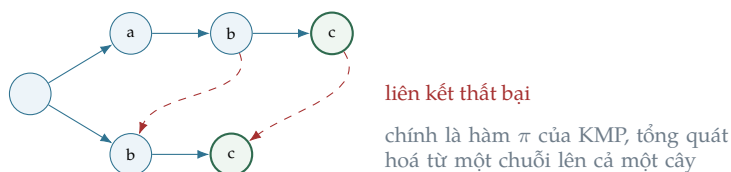
Hàm π còn cho thêm hai kết quả đáng nhớ, và chúng thường là mục đích thật sự của bài chứ không phải việc so khớp: *chu kỳ nhỏ nhất* của chuỗi bằng $n - \pi[n]$ khi n chia hết cho hiệu đó, và *mọi biên* của chuỗi tìm được bằng cách lặp $\pi[n], \pi[\pi[n]], \dots$

1.2 Aho–Corasick: KMP cho nhiều mẫu cùng lúc

Nếu có k mẫu, chạy KMP k lần cho $O(k \cdot n)$. Aho–Corasick làm trong một lượt quét văn bản.

Ý tưởng ghép hai thứ đã biết: xếp mọi mẫu vào một *trie*, rồi thêm vào mỗi nút một “liên kết thất bại” trở tới nút ứng với hậu tố dài nhất của chuỗi hiện tại mà cũng là một tiền tố của mẫu nào đó. Nói cách khác: liên kết thất bại là hàm π của KMP, tổng quát hoá từ một chuỗi lên cả một cây. Khi quét văn bản, ta đi trên *trie*; gặp ký tự không có cạnh thì theo liên kết thất bại, hết như KMP nhảy về $\pi[k]$.

Kết quả là $O(n + \text{tổng độ dài mẫu} + \text{số lần khớp})$. Đây là thuật toán chạy phía sau các công cụ tìm nhiều từ khoá trong một lượt — lệnh tìm nhiều mẫu của Unix, các hệ phát hiện xâm nhập quét gói tin theo hàng nghìn chữ ký, các bộ lọc nội dung.



Hình 1.2. Automaton Aho–Corasick cho tập mẫu $\{abc, bc\}$. Các mẫu chia sẻ tiền tố nên đi chung nhánh; liên kết thất bại (nét đứt) cho phép chuyển trạng thái khi lệch mà không đọc lại văn bản. Nhờ vậy chi phí quét không phụ thuộc số mẫu.

1.3 Băm chuỗi: lựa chọn ngắn với cái giá xác suất

Băm đa thức (B/08) cho phép so sánh hai đoạn con bất kỳ trong $O(1)$ sau tiền xử lý $O(n)$. Với công cụ đó, rất nhiều bài chuỗi sụp xuống thành dễ: tìm mẫu, tìm chuỗi con lặp lại dài nhất (kết hợp nhị phân trên độ dài, C/19), so sánh hai đoạn, đếm chuỗi con phân biệt.

Ba điều phải nhớ mỗi lần dùng, đã nói ở chương B/08 nhưng đáng nhắc lại vì đây là nơi chúng gây hại nhiều nhất: chọn cơ sở *ngẫu nhiên lúc chạy*; dùng hai modulo khi số chuỗi cần so lớn hơn cỡ $3 \cdot 10^4$; và nhớ rằng đây là thuật toán xác suất.

Vậy vì sao vẫn học KMP? Ba lý do. Thứ nhất, KMP cho kết quả *chắc chắn đúng*. Thứ hai, hàm π cho thông tin về cấu trúc chuỗi — biên, chu kỳ — mà băm không cho. Thứ ba, và quan trọng nhất về mặt học tập: ý tưởng của KMP — dùng lại thông tin của lần thất bại — là ý tưởng tổng quát, và bạn sẽ gặp lại nó ở Aho–Corasick, ở các thuật toán tự động hoá, và ở nhiều nơi ngoài chuỗi.

1.4 Nhìn được mọi chuỗi con: trie, mảng hậu tố, automaton

Các công cụ trên trả lời câu hỏi về *một mẫu cho trước*. Có một lớp câu hỏi khác đòi hỏi nhiều hơn: “chuỗi con lặp lại dài nhất là gì”, “có bao nhiêu chuỗi con phân biệt”, “hai chuỗi có đoạn chung dài nhất là bao nhiêu”. Những câu này cần một cấu trúc *nhìn được toàn bộ tập chuỗi con* cùng lúc.

Trie là bước đầu: lưu một tập chuỗi thành cây theo ký tự. Nó cho tra cứu theo tiền tố, tự động hoàn thành, và là nền của Aho–Corasick. Nhược điểm: bộ nhớ lớn nếu bảng chữ cái rộng.

Mảng hậu tố là công cụ chủ lực. Ý tưởng: lấy toàn bộ n hậu tố của chuỗi và sắp chúng theo thứ tự từ điển; mảng hậu tố lưu vị trí bắt đầu của chúng theo thứ tự đó. Kèm theo là mảng LCP, ghi độ dài tiền tố chung của hai hậu tố liền nhau trong thứ tự đã sắp. Xây được trong $O(n \log n)$ hoặc $O(n)$.

Vì sao nó “giải quyết một họ bài”? Vì sau khi có mảng hậu tố và LCP, các câu hỏi về chuỗi con trở thành các câu hỏi về *đoạn trên một mảng số* — thứ mà cả phần B trước đó đã trang bị đầy đủ công cụ. Chuỗi con lặp lại dài nhất chính là giá trị lớn nhất trong mảng LCP. Số chuỗi con phân biệt là tổng độ dài các hậu tố trừ tổng LCP. Chuỗi con chung dài nhất của hai chuỗi giải bằng cách nối

hai chuỗi và tìm cực đại LCP giữa hai hậu tố khác nguồn gốc. Nghĩa là mảng hậu tố đóng vai trò một *phép biến đổi*: nó chuyển bài toán từ miền chuỗi sang miền mảng, nơi ta đã mạnh.

Suffix automaton và *suffix tree* là hai cấu trúc mạnh hơn, biểu diễn được mọi chuỗi con trong $O(n)$ trạng thái và cho nhiều bài lời giải tuyến tính đẹp. Chúng cũng khó cài và khó nhớ hơn hẳn. Lời khuyên thực dụng: nắm chắc mảng hậu tố trước; *suffix automaton* xếp vào loại tham khảo, học khi gặp bài thật cần.

1.5 Manacher và một mẫu tư duy đáng nhớ

Bài tìm chuỗi con đối xứng dài nhất có lời giải $O(n)$ của Manacher, và ý tưởng của nó là một mẫu dùng lại được: *tái sử dụng đối xứng đã biết*. Khi ta đã biết một chuỗi đối xứng lớn quanh tâm c , thì với một vị trí i nằm trong nó, ta biết trước phần lớn thông tin về bán kính đối xứng tại i — vì nó phải giống với bán kính tại vị trí đối xứng của i qua c , ít nhất cho tới khi chạm biên.

Mẫu này giống hệt cách tính hàm Z (tái sử dụng đoạn khớp phải nhất) và giống cách Aho–Corasick tái dùng liên kết thất bại. Nếu bạn muốn nhớ một điều duy nhất về nhóm thuật toán chuỗi tuyến tính, hãy nhớ: *luôn duy trì cấu trúc phải nhất đã biết, và dùng nó để bắt đầu chỗ mới không phải từ số không*.

Công cụ	Xây	Trả lời	Chọn khi
Băm chuỗi	$O(n)$	so sánh đoạn $O(1)$	Cần nhanh và ngắn; chấp nhận xác suất
KMP / hàm Z	$O(n + m)$	so khớp một mẫu	Cần chắc chắn đúng; cần biên và chu kỳ
Aho–Corasick	$O(\sum P_i)$	nhiều mẫu, một lượt	Hàng nghìn mẫu cố định
Trie	$O(\sum S_i)$	tra theo tiền tố	Tự động hoàn thành, từ điển
Mảng hậu tố + LCP	$O(n \log n)$	mọi câu hỏi về chuỗi con	Bài về chuỗi con lặp/chung/phân biệt
Suffix automaton	$O(n)$	mọi chuỗi con, đếm	Khi mảng hậu tố không đủ; tham khảo

1.6 Gốc rễ: từ tìm kiếm thư mục tới giải mã bộ gen

Bài toán so khớp chuỗi có một lịch sử gắn liền với những nhu cầu rất thực tế của từng thời kỳ, và mỗi thời kỳ lại sinh ra một cấu trúc.

Thuật toán KMP được công bố năm 1977[1], nhưng câu chuyện đằng sau đáng nhớ: Morris và Pratt tìm ra thuật toán trước, còn Knuth đến với nó từ hướng lý thuyết — ông đang nghiên cứu một kết quả về automaton hai chiều và nhận ra nó kéo theo một thuật toán so khớp tuyến tính. Ba người ghép công trình lại. Điều đáng học ở đây là hai con đường rất khác nhau — kỹ thuật và lý thuyết — dẫn tới cùng một thuật toán, và bài báo chung mạnh hơn cả hai.

Aho và Corasick công bố thuật toán của họ năm 1975[2] với một bài toán rất cụ thể trong tiêu đề: tra cứu thư mục thư viện. Nhu cầu là quét một khối văn bản để tìm hàng nghìn từ khoá cùng lúc — và cấu trúc họ tạo ra sau đó trở thành công cụ chuẩn cho việc quét mẫu trong các hệ thống an ninh mạng, nơi mỗi gói tin phải được đối chiếu với hàng nghìn chữ ký tấn công.

Nhánh cấu trúc hậu tố có một câu chuyện về đánh đổi bộ nhớ rất đáng kể. Cây hậu tố được Weiner đưa ra năm 1973 và cho lời giải tuyến tính cho rất nhiều bài, nhưng nó tốn bộ nhớ lớn — hằng số cao và phụ thuộc bảng chữ cái. Năm 1990, Manber và Myers[3] đề xuất mảng hậu tố với

động cơ rõ ràng là *tiết kiệm bộ nhớ*: thay cây bằng một mảng số nguyên. Đây là ví dụ mẫu mực cho việc một cấu trúc “yếu hơn” về lý thuyết lại thắng trong thực tế nhờ hằng số.

Và cú hích lớn nhất cho cả nhánh này đến từ ngoài ngành: dự án giải mã bộ gen người. Việc căn chỉnh hàng tỉ đoạn đọc DNA lên một bộ gen tham chiếu là bài toán so khớp chuỗi ở quy mô chưa từng có, với ràng buộc bộ nhớ khắc nghiệt. Điều đó thúc đẩy các chỉ mục nén dựa trên phép biến đổi Burrows–Wheeler, cho phép tìm kiếm trên bộ gen ba tỉ ký tự trong vài gigabyte bộ nhớ — và các công cụ căn chỉnh phổ biến trong tin sinh học ngày nay đều dựa trên đó.

I Gốc rễ: ý tưởng này ra đời từ bài toán nào

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó
1970–77 — Morris, Pratt, Knuth	So khớp một mẫu trong văn bản mà không lùi con trỏ văn bản.	KMP; hai con đường (kỹ thuật và lý thuyết automaton) gặp nhau ở cùng một thuật toán.
1975 — Aho & Corasick	Tra cứu thư mục thư viện: tìm hàng nghìn từ khoá trong một lượt quét.	Trie + liên kết thất bại; nay là lõi của quét chữ ký trong hệ thống an ninh mạng.
1973 — Weiner	Cần trả lời mọi câu hỏi về chuỗi con trong thời gian tuyến tính.	Cây hậu tố: mạnh nhưng tốn bộ nhớ, hằng số cao, phụ thuộc bảng chữ cái.
1990 — Manber & Myers	Cây hậu tố quá tốn bộ nhớ cho dữ liệu lớn thời đó.	Mảng hậu tố: đổi cấu trúc cây lấy một mảng số nguyên; ví dụ điển hình cho việc hằng số thắng lý thuyết.
1994 — Burrows & Wheeler	Nén văn bản tốt hơn bằng cách sắp xếp lại ký tự cho dễ nén.	Phép biến đổi BWT; sau này thành nền cho chỉ mục nén tìm kiếm được.
Thập niên 2000 — tin sinh học	Căn chỉnh hàng tỉ đoạn đọc DNA lên bộ gen ba tỉ ký tự với bộ nhớ hạn chế.	Chỉ mục FM dựa trên BWT; các công cụ căn chỉnh chuẩn của ngành đều dùng, đưa thuật toán chuỗi vào ứng dụng quy mô lớn nhất của nó.

I 1.7 Liên hệ ngang

I Liên hệ ngang: ý này còn sống ở đâu nữa

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu
Tin sinh học	Căn chỉnh đoạn đọc DNA, tìm mẫu trong bộ gen	Cùng bài toán, khác quy mô và khác ràng buộc: bảng chữ cái chỉ bốn ký tự, nhưng chuỗi dài ba tỉ và phải chịu được sai khác. Đây là lý do ngành này đẩy mạnh chỉ mục nén, thứ mà lập trình thi đấu ít gặp.
Nén dữ liệu	LZ77, BWT	Nén tìm các đoạn lặp lại — đúng thông tin mà mảng hậu tố và LCP mô tả. Liên hệ hai chiều: cấu trúc chuỗi giúp nén, và phép biến đổi phục vụ nén (BWT) quay lại thành chỉ mục tìm kiếm.
An ninh mạng	Hệ phát hiện xâm nhập quét gói tin theo hàng nghìn chữ ký	Aho–Corasick nguyên bản. Ràng buộc thêm so với bài tập: phải chạy ở tốc độ đường truyền, nên bộ nhớ của automaton và tính thân thiện cache là yếu tố quyết định.
Trình biên dịch	Phân tích từ vựng bằng automaton hữu hạn	Aho–Corasick là một automaton nhận biết tập mẫu; bộ phân tích từ vựng cũng là automaton. Cùng ý tưởng: biên dịch tập mẫu thành một máy trạng thái rồi quét văn bản một lượt.

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu (tiếp)
Công cụ tìm kiếm	Chỉ mục ngược, gợi ý tự động, sửa lỗi chính tả	Trie cho gợi ý theo tiền tố; khoảng cách chỉnh sửa (C/16) cho sửa lỗi. Khác quy mô: chỉ mục thật phân tán và nén, nhưng ý tưởng gốc vẫn là các cấu trúc ở đây.
Xử lý ngôn ngữ tự nhiên	Tách từ con (subword tokenization) bằng từ điển và trie	Bộ tách từ của mô hình ngôn ngữ dùng cấu trúc kiểu trie để khớp tham lam đoạn dài nhất. Cùng cấu trúc, khác mục tiêu: ở đây là biểu diễn đầu vào chứ không phải tìm kiếm.
Thị giác máy tính, SLAM	So khớp mô tả nhị phân, phát hiện lặp lại chuỗi ảnh	Không dùng KMP, nhưng chung tinh thần: chuyển bài so khớp về một biểu diễn cho phép so sánh $O(1)$ (băm, mã nhị phân) rồi kiểm chứng lại bằng bước đắt hơn. Kiến trúc “lọc rẻ rồi kiểm đắt” giống hệt mục 3.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Thông tin về *phần đã khớp*. Khi lệch sau k ký tự khớp, ta biết chính xác k ký tự vừa qua của văn bản bằng $P[1..k]$; cách này vớt bỏ điều đó và đọc lại từ vị trí kế tiếp. KMP dùng nó để xác định vị trí thử tiếp theo duy nhất còn cơ hội — vị trí ứng với biên dài nhất của $P[1..k]$ — nên con trỏ văn bản không bao giờ lùi. Đây là lần thứ ba trong phần B xuất hiện lập luận “mỗi chỉ số chỉ đi một chiều”.
- A2. Ba lý do. Một, KMP đúng đắn chắc chắn, còn băm có xác suất sai — nhỏ nhưng khác không, và nó tăng theo bình phương số phép so sánh. Hai, hàm π cho thông tin cấu trúc mà băm không cho: mọi biên, chu kỳ nhỏ nhất — và nhiều bài thật ra hỏi đúng những thứ này chứ không hỏi việc so khớp. Ba, ý tưởng “dùng lại thông tin của lần thất bại” là ý tưởng tổng quát, quay lại ở Aho–Corasick và ở các thuật toán tự động hoá khác.
- A3. Vì nó là một *phép biến đổi* miền bài toán: sau khi có mảng hậu tố và mảng LCP, các câu hỏi về chuỗi con trở thành câu hỏi về đoạn trên một mảng số nguyên — miền mà cả phần B đã trang bị đầy đủ công cụ. Chuỗi con lặp lại dài nhất thành cực đại của LCP; số chuỗi con phân biệt thành một hiệu của hai tổng; chuỗi chung dài nhất thành truy vấn trên đoạn. Một cấu trúc mở khoá cả một họ bài chính là vì lý do đó.
- A4. Khác ở chỗ nó *biên dịch toàn bộ tập mẫu thành một máy trạng thái* trước, rồi quét văn bản đúng một lần với chi phí không phụ thuộc số mẫu. Điều làm việc đó khả thi là các mẫu chia sẻ tiền tố với nhau — trie khai thác đúng phần chia sẻ ấy — và liên kết thất bại cho phép chuyển trạng thái khi lệch mà không cần quay lui. Chạy KMP k lần thì mỗi lần đọc lại toàn bộ văn bản, tức là bỏ qua chính phần chia sẻ đó.
- A5. Tái sử dụng đối xứng đã biết. Nếu đã biết một chuỗi đối xứng lớn quanh tâm c , thì với vị trí i nằm bên trong nó, bán kính đối xứng tại i ít nhất bằng bán kính tại điểm đối xứng của i qua c , cho tới khi chạm biên của chuỗi lớn. Nhờ đó phần lớn vị trí được khởi tạo sẵn thay vì bắt đầu từ số không, và tổng chi phí mở rộng là tuyến tính. Cùng mẫu với cách tính hàm Z: luôn duy trì cấu trúc phải nhất đã biết.

NÓI LẠI THẬT ĐƠN GIẢN

Giả sử bạn đang dò xem từ “ababac” có xuất hiện trong một trang sách hay không. Cách vụng là đặt từ đó lên đầu dòng, so từng chữ; hễ sai một chữ thì dịch sang phải một ô rồi so lại từ đầu. Cách này phí ở chỗ: khi bạn đã so đúng năm chữ rồi mới sai, bạn *đã biết* năm chữ đó của trang sách là gì — vậy mà bạn vứt hết và dò lại.

KMP chỉ làm một việc: trước khi dò, nó đọc kỹ chính cái từ cần tìm và ghi lại một bảng nhỏ trả lời câu “nếu tôi khớp được k chữ rồi sai, thì phải trượt sang bao nhiêu ô là hợp lý”. Câu trả lời phụ thuộc vào chỗ từ đó tự lặp lại chính nó ở đầu và cuối — ví dụ “ababa” có phần “aba” vừa là đầu vừa là đuôi, nên trượt hai ô là đủ, không cần trượt một ô. Nhờ bảng đó, mắt bạn không bao giờ phải đọc lại một chữ nào của trang sách.

Nếu cần tìm hàng nghìn từ cùng lúc thì đừng dò từng từ một. Hãy xếp tất cả các từ vào một cái cây theo chữ cái đầu — những từ có chung phần đầu thì đi chung một nhánh — rồi đọc trang sách một lượt và đi trên cây đó. Đó là Aho–Corasick.

Cuối cùng, có một cách nhìn khác cho các câu hỏi kiểu “đoạn nào lặp lại nhiều nhất”: hãy liệt kê mọi phần đuôi của chuỗi rồi xếp chúng theo thứ tự từ điển, như xếp từ trong từ điển. Khi đó những đoạn giống nhau tự nằm cạnh nhau, và câu hỏi “đoạn lặp dài nhất” chỉ còn là “hai hàng xóm nào giống nhau nhiều chữ nhất”. Đó là mảng hậu tố, và giá trị của nó là biến một bài về chuỗi thành một bài về dãy số thông thường.

Một câu: Mọi thuật toán chuỗi nhanh đều trả lời cùng một câu: khi thất bại, tôi đã biết được gì để không phải làm lại từ đầu?

Chỗ không rút gọn được: Cấu trúc hậu tố (mảng hậu tố, automaton) không có cách giải thích nào thật sự ngắn gọn; phải cài một lần và chạy tay trên ví dụ nhỏ mới thấy được.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Ý chung của mọi thuật toán chuỗi tuyến tính	Khi thất bại tại vị trí i , khai thác thông tin đã biết thay vì bắt đầu lại; duy trì “cấu trúc phải nhất đã biết” và dùng nó khởi tạo cho vị trí mới.
••• Biên và hàm π	Biên là chuỗi vừa là tiền tố vừa là hậu tố thực sự; $\pi[i]$ là biên dài nhất của tiền tố kết thúc tại i . Toàn bộ tiền xử lý của KMP.
••• Vì sao KMP là $O(n + m)$	Con trỏ văn bản chỉ tiến; con trỏ mẫu tăng tối đa n lần nên giảm tối đa n lần. Cùng lập luận với hai con trỏ và ngăn xếp đơn điệu.
••• Chu kỳ nhỏ nhất từ π	Bằng $n - \pi[n]$ khi n chia hết cho hiệu đó; mọi biên tìm bằng cách lặp $\pi[n], \pi[\pi[n]], \dots$
••• Mẹo dùng chung của KMP và hàm Z	Tính hàm trên chuỗi ghép $P + \# + T$; vị trí có giá trị bằng m là một lần xuất hiện.
••• Mảng hậu tố giải quyết gì	Biến câu hỏi về chuỗi con thành câu hỏi về đoạn trên mảng số: chuỗi lặp dài nhất = $\max \text{LCP}$; số chuỗi con phân biệt = tổng độ dài trừ tổng LCP.
•• Aho–Corasick	Trie của mọi mẫu + liên kết thất bại (chính là π tổng quát hoá lên cây); quét văn bản một lượt, chi phí không phụ thuộc số mẫu.
•• Ba lưu ý khi băm chuỗi	Cơ số ngẫu nhiên lúc chạy; hai modulo khi số chuỗi vượt cỡ $3 \cdot 10^4$; luôn nhớ đây là thuật toán xác suất.
•• Vì sao vẫn học KMP dù có băm	Chắc chắn đúng; cho thông tin cấu trúc (biên, chu kỳ); và ý tưởng “dùng lại lần thất bại” tổng quát hơn nhiều so với một thuật toán.
•• Manacher	Tái sử dụng đối xứng: bán kính tại i khởi tạo từ bán kính tại điểm đối xứng qua tâm của chuỗi đối xứng phải nhất đã biết.
•• Trie	Cây theo ký tự, mỗi đường là một tiền tố; dùng cho từ điển, gợi ý tự động, và làm nền cho Aho–Corasick.
• Suffix automaton / suffix tree	Biểu diễn mọi chuỗi con trong $O(n)$ trạng thái; mạnh hơn mảng hậu tố nhưng khó cài — học sau khi đã chắc mảng hậu tố.

••• phải thuộc •• nên nắm • biết là có

Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

1. Tôi nên học KMP hay hàm Z trước?

| Hàm Z, vì định nghĩa của nó trực tiếp hơn — “đoạn khớp với tiền tố bắt đầu tại i dài bao nhiêu” — và cách tính dễ hình dung bằng hình. KMP có định nghĩa gián tiếp hơn qua khái niệm biên, nhưng

lại cần thiết khi bài hỏi về chu kỳ, về biên, hoặc khi cần xây automaton. Trong thực hành thi đấu, nhiều người dùng hàm Z cho việc so khớp và chỉ dùng π khi bài hỏi cấu trúc.

2. Khi nào tôi thực sự cần mảng hậu tố thay vì băm chuỗi?

| Khi bài hỏi về *tồn bộ tập chuỗi con* chứ không về một mẫu cụ thể: đếm số chuỗi con phân biệt, sắp xếp các chuỗi con, tìm chuỗi con thứ k theo thứ tự từ điển. Băm giải tốt các bài “so sánh hai đoạn” và nhiều bài tìm kiếm có kết hợp nhị phân, nhưng nó không cho bạn một thứ tự trên tập chuỗi con. Quy tắc thực dụng: đề có chữ “phân biệt”, “thứ k theo từ điển”, “sắp xếp các chuỗi con” thì nghĩ tới mảng hậu tố.

3. Xây mảng hậu tố $O(n \log^2 n)$ có đủ dùng không?

| Hầu như luôn đủ với n tới vài trăm nghìn, và nó ngắn hơn hẳn bản $O(n \log n)$ dùng sắp xếp đếm. Trong bài mà n tới 10^6 , hãy dùng bản $O(n \log n)$. Bản tuyến tính đẹp về lý thuyết nhưng dài và ít khi cần — đúng tinh thần “chọn cấu trúc yếu nhất còn đủ dùng”.

4. Aho–Corasick tốn bộ nhớ thế nào khi bảng chữ cái lớn?

| Nếu mỗi nút lưu một mảng chuyển trạng thái đầy đủ thì bộ nhớ là số nút nhân kích thước bảng chữ cái — với bảng Unicode thì không khả thi. Ba cách xử lý: dùng bảng băm cho các cạnh thay vì mảng; chỉ lưu cạnh thật và tra liên kết thất bại khi thiếu (chậm hơn một chút nhưng gọn hơn nhiều); hoặc ánh xạ bảng chữ cái về tập ký tự thực sự xuất hiện. Trong hệ thống thật, lựa chọn này quyết định việc automaton có nằm vừa trong cache hay không, và đó mới là yếu tố chi phối tốc độ.

5. Bài so khớp có cho phép sai khác vài ký tự thì dùng gì?

| Đó là bài toán khác hẳn và các công cụ trong chương này không giải trực tiếp. Với sai khác nhỏ, hướng chuẩn là quy hoạch động khoảng cách chỉnh sửa ($C/16$), có thể tăng tốc bằng cách chỉ tính một dải quanh đường chéo khi số sai khác bị chặn. Với dữ liệu rất lớn như bộ gen, người ta dùng chiến lược “hạt giống rồi mở rộng”: tìm các đoạn khớp chính xác ngắn bằng chỉ mục, rồi mở rộng bằng căn chỉnh có chi phí — lại đúng kiến trúc lọc rẻ trước, kiểm đất sau.

6. Vì sao ký tự phân cách # trong mẹo $P + \# + T$ lại quan trọng?

| Vì nếu không có nó, một đoạn khớp có thể vắt qua ranh giới giữa mẫu và văn bản, tạo ra kết quả sai. Ký tự phân cách phải *không xuất hiện* trong cả mẫu lẫn văn bản, và trong bài dùng bảng chữ cái đầy đủ thì bạn phải chọn một giá trị ngoài bảng — đây là lỗi hay gặp khi bài cho phép mọi byte làm ký tự.

7. Có mối liên hệ nào giữa chương này và automaton trong lý thuyết tính toán không?

| Có, và nó khá trực tiếp. Hàm π của KMP chính là hàm chuyển trạng thái của một automaton hữu hạn nhận biết mẫu; Aho–Corasick là automaton nhận biết một tập mẫu; suffix automaton là automaton nhận biết mọi hậu tố của chuỗi. Nhìn theo góc này, cả chương là một chuỗi các cách xây automaton nhỏ gọn cho những ngôn ngữ rất cụ thể — và đó cũng là con đường mà Knuth đã đi tới KMP.

| Bài tập (mức: ● khởi động ●● chuẩn ●●● khó)

Bài	Nguồn	Điều cần nhận ra
● Tìm mọi vị trí xuất hiện của một mẫu	CSES “String Matching”[4]	Làm bằng cả ba cách: băm, hàm Z, KMP; so độ dài code và thời gian chạy.
● Tính hàm tiền tố và liệt kê mọi biên của chuỗi	Bài kinh điển	Lập $\pi[n], \pi[\pi[n]], \dots$; kiểm chứng bằng tay trên chuỗi ngắn.
● Kiểm tra một chuỗi có phải là phép quay của chuỗi kia	Bài kinh điển	Nối chuỗi với chính nó rồi so khớp — một mẹo đáng nhớ.
●● Chu kỳ nhỏ nhất của chuỗi	CSES nhóm bài chuỗi[4]	$n - \pi[n]$ khi chia hết; hãy tự chứng minh điều kiện chia hết là cần thiết.
●● Đếm số lần xuất hiện của mỗi mẫu trong một tập mẫu	Bài kinh điển	Aho–Corasick; cộng dồn theo cây liên kết thất bại.

Bài	Nguồn	Điều cần nhận ra (tiếp)
•• Chuỗi con đối xứng dài nhất	LeetCode “Longest Palindromic Substring”	Manacher; hoặc băm cộng nhị phân trên bán kính — so hai cách.
•• Tiền tố chung dài nhất của một tập chuỗi	Bài kinh điển	Trie hoặc so trực tiếp; bài đơn giản nhưng luyện thói quen xét trường hợp tập rỗng.
••• Chuỗi con lặp lại dài nhất	Bài kinh điển	Mảng hậu tố + LCP, hoặc băm + nhị phân; hai cách cùng $O(n \log n)$ với hằng số rất khác.
••• Đếm số chuỗi con phân biệt	CSES “Distinct Substrings”-loại[4]	Tổng độ dài hậu tố trừ tổng LCP; tự dẫn ra công thức trước khi tra.
••• Chuỗi con chung dài nhất của hai chuỗi	Bài kinh điển	Nối hai chuỗi với ký tự phân cách; cực đại LCP giữa hai hậu tố khác nguồn.
••• Chuỗi con thứ k theo thứ tự từ điển	Bài thi đấu kinh điển	Bài mà băm không giải được — lý do rõ ràng nhất để học mảng hậu tố hoặc suffix automaton.
••• Cài Aho–Corasick và dùng nó lọc một tập từ khoá trên văn bản lớn	Bài tập ứng dụng	Đo thời gian so với chạy KMP nhiều lần; số liệu sẽ cho thấy khác biệt rõ hơn mọi lập luận.

Kết chương và đọc tiếp

Phần B khép lại ở đây. Bảy chương vừa qua đều trả lời cùng một câu hỏi — *dữ liệu nên nằm ở dạng nào để thao tác tôi lập lại nhiều nhất trở nên rẻ* — và đều tuân theo cùng một nguyên tắc: chọn cấu trúc duy trì *đúng* lượng trật tự mà bài cần, không hơn. Heap biết ít hơn cây nên rẻ hơn; DSU biết ít hơn đồ thị nên gần như miễn phí; băm bỏ thứ tự nên nhanh hơn cây. Mỗi khả năng bạn đòi thêm đều có giá.

Phần C đổi câu hỏi lần nữa, và đây là câu hỏi trung tâm của cả quyển sách: *dùng khuôn tư duy nào để sinh ra lời giải*. Nó bắt đầu từ chỗ mọi lời giải bắt đầu — vết cạn — và chỉ ra rằng phần lớn thuật toán hay chỉ là vết cạn cộng thêm một quan sát cho phép bỏ bớt nhánh (C/13).

Đọc thêm: công trình gốc (chỉ liệt kê nguồn đã qua phản biện; bản tiền ấn được ghi rõ)

Mức	Công trình và tác giả	Nơi công bố, năm	Vì sao đọc, đọc phần nào
nền tảng	Knuth, Morris, Pratt, “Fast Pattern Matching in Strings”	SIAM Journal on Computing, 1977	Bài gốc của KMP. Đọc phần xây hàm thất bại — chỗ mà mọi tài liệu về sau đều rút gọn quá tay.
nền tảng	Aho & Corasick, “Efficient String Matching: An Aid to Bibliographic Search”	Communications of the ACM, 1975	Động cơ ban đầu là tra cứu thư mục; đọc để thấy vì sao cấu trúc phải là trie cộng liên kết thất bại.
tra khi cần	Manber & Myers, “Suffix Arrays: A New Method for On-line String Searches”	SIAM Journal on Computing, 1993	Bài đưa mảng hậu tố thành lựa chọn thay cây hậu tố; đọc phần so sánh bộ nhớ.
tra khi cần	Kärkkäinen, Sanders, Burkhardt, “Linear Work Suffix Array Construction”	Journal of the ACM, 2006	Thuật toán skew: dựng mảng hậu tố trong $O(n)$ bằng chia để trị theo phần dư của chỉ số. Đọc khi cần bản tuyến tính thay cho bản $O(n \log n)$.
tra khi cần	Ukkonen, “On-line Construction of Suffix Trees”	Algorithmica, 1995	Bản dựng cây hậu tố trực tuyến; tra khi cần chứ không cần thuộc.

Nguồn

- [1] D. E. Knuth, J. H. Morris, and V. R. Pratt, “Fast Pattern Matching in Strings,” *SIAM Journal on Computing*, vol. 6, no. 2, pp. 323–350, 1977.
- [2] A. V. Aho and M. J. Corasick, “Efficient String Matching: An Aid to Bibliographic Search,” *Communications of the ACM*, vol. 18, no. 6, pp. 333–340, 1975.
- [3] U. Manber and G. Myers, “Suffix Arrays: A New Method for On-line String Searches,” *SIAM Journal on Computing*, vol. 22, no. 5, pp. 935–948, 1993.
- [4] A. Laaksonen. “CSES Problem Set.” (2026), address: <https://cses.fi/problemset/> (visited on 09/06/2026).